

BEANS

BEANS (*student proposal*)

Assessment: ✓ Fit · **Category:** Visualization & Analytics · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval · **Runner-up category:** Education + Training

Proposal (as submitted): *BEANS* is a learning-oriented coffee databank web-app that educates newcomers on beans and preparation methods by preference. It cross-references brew details (bean type, grind size, method) to **estimate the final taste**; users log a brew + feedback, and the app suggests improvements/alternatives from the **collective community databank**. A relevancy search recommends coffees by preference + search history; there's a shop for beans/gear, monthly subscription tiers, and an inventory tracker (with nearby store/cafe suggestions). Named technical features: a **Parametric Taste & Extraction Visualizer** (brew variables → dynamic extraction-yield curves + a sensory taste-profile **radar chart**); a **Brew Diagnostics & Dial-In Tool** (qualitative feedback like *sour/astringent/hollow* → inverse parameter corrections); a **Taste-Profile Catalog Search & Vector Ranking** (multi-attribute search ranking beans by similarity to the user's taste + logs); and a **high-throughput C++ vector search + neural re-ranking engine**. Load-bearing C++: a compiled **C++→WebAssembly** module runs client-side computing particle-level solute diffusion / extraction dynamics in a **fixed-timestep numerical loop**, with slider inputs continuously re-injecting parameters to update the taste radar without frame drops or network hops; plus a **dedicated C++ service** for high-dimensional vector math (hardware SIMD distance across flavor profiles) and an **embedded lightweight neural cross-encoder** that re-ranks candidate beans before the API returns results. Tools: a **Dial-In & Recipe Calibration Utility** (author/test/tune brew parameters per bean; roasters author bean-specific templates with grind/temperature/ratio bounds + sensory target envelopes; a closed-loop diagnostic that ingests taste feedback, runs inverse heuristics, and previews the corrected radar before committing a versioned profile). Design features: estimate brew taste from inputs; a guide-me/

recipe feature; a community to share experiments; and monetization (B2C bean/machine subscriptions; B2B roaster onboarding).

Note on category: filed under **Visualization & Analytics** — the load-bearing product is the **interactive, data-moving visualization** (extraction curves + taste radar recomputed live from slider inputs) plus **vector-ranking analytics** over a coffee databank; coffee brewing isn't claimed by another category, and the data moves (live recompute, ranking). Runner-up **Education + Training** (the "learning-oriented / guide-me" framing) — but there's no structured curriculum/competency, the mechanism is visualization + recommendation, so V&A is the better home. (Not CMS: roaster authoring is one tool, not the product; not Simulator: the extraction sim feeds the visualization, it isn't the whole product.)

Scope & risk notes (advisory, not blocking):

1. **Keep the load-bearing C++ on the team's side of the AI.** The graded performant-C++ is the **extraction WASM solver**, the **SIMD vector-search** and the custom chart rendering — the "lightweight neural cross-encoder" should be a **small, bounded component** behind the C++ vector-search service (with the ranking working from the vector similarity even if the neural re-rank is swapped out), not the deliverable itself.
2. **Don't let the e-commerce become the product.** The shop / subscription / B2B-onboarding / inventory features are legitimate but should stay **light backend** (Firebase + the databank) — the visualizer, the dial-in tool and the taste-vector ranking are the module deliverables; otherwise the project risks collapsing into CRUD + a store.

References:

- [Barista Hustle — "The Coffee Compass"](#) (an interactive diagnostic that maps qualitative faults — sour, bitter, weak, strong — to concrete brew-parameter corrections: "extract more/less", "more/less coffee"; the direct precedent for the Dial-In & Diagnostics tool)
- [Coffee extraction — Wikipedia](#) (the extraction-yield vs strength / TDS model and the brewing-control chart that relate grind/ratio/time/temperature to taste — the domain model the parametric visualizer estimates)
- [SCA — "How to Use the Coffee Taster's Flavor Wheel"](#) (the standardized multi-attribute sensory lexicon/vocabulary behind the taste-profile radar + the vector matching)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	BEANS
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Web browser on mobile — the stated architecture is a C++ → WASM client (the extraction solver runs client-side in the browser), so the app is web-first and mobile-web friendly (device: Pixel 9a); the Native PC build is the development/tools vehicle (the dial-in calibration utility). **Extended:** Firebase (accounts, community databank, shop/subscription data) + a Linux server the team sets up by M2 (full use T2) hosting the dedicated C++ vector-search + neural re-ranking service.

Front end: Custom-rendered data views — the extraction-yield curves and the sensory ****taste-profile radar****, recomputed live from slider inputs — plus

the catalog/search results, the dial-in diagnostic view, community records and the shop UI. **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten) for the solver + chart rendering (OpenGL ES / WebGL for the charts).

Back end: The client-side WASM extraction solver + the custom chart renderer, and a server-side **C++ vector-search service** (SIMD distance across flavor profiles + a bounded neural re-rank) over the coffee databank; plus Firebase for accounts, community logs and shop/subscription data.

Languages: C++ (solver + vector service); Firebase-backed services + the Linux vector-service host on the Extended tier.

Backend scope: Server-indicated (T2) — the vector-search service + databank + shop seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* two load-bearing pieces — the client-side **fixed-timestep WASM extraction solver** (particle-level solute diffusion → dissolved solids / extraction yield → taste coordinates, recomputed live as sliders change) **and** the server-side **C++ vector-search service** (SIMD multi-attribute distance + a bounded neural re-rank) ranking beans by taste similarity.
- *Data-driven:* beans, grind/temperature/ratio parameters, sensory attribute vocabularies, dial-in templates + sensory envelopes, and community logs are all data — a new bean or dial-in template is a data change (authored via the calibration tool).
- *2D:* the extraction-yield curves and the taste-profile radar are **custom-rendered charts** (no chart libraries) recomputed in real time — this is the 2D gate; plus the catalog/diagnostic/shop UI. *3D:* not needed. *Networking:* the vector service + shop are async client↔server (Application Communication ✓), so a bespoke streaming Networking System is optional (●).
- *Custom tool:* the **Dial-In & Recipe Calibration Utility** — roasters author bean templates (grind/temperature/ratio bounds + sensory envelopes) and the closed-loop diagnostic previews the corrected radar before committing a versioned profile.
- *External DB:* the coffee databank, community logs, shop/subscription and inventory. *Auth »:* accounts (B2C users + B2B roasters). *Analytics »:* optional — the recommendation/search-history analytics (the team's own ranking is the deliverable via G10).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Visualization & Analytics pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the fixed-timestep **extraction solver** (WASM).

Tech & dependencies

- **Tech: Web build:** A C++17/20 fixed-timestep numerical loop computing solute diffusion / dissolved solids / extraction yield → taste coordinates, compiled to WASM (Emscripten) and driven live by slider inputs without network hops; CMake, Git.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** M1G5 (the charts it feeds), M1G9 (slider interaction), the dial-in tool.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** A data model for beans, brew parameters, sensory attribute vocabularies, dial-in templates + sensory envelopes and community logs, in JSON, hot-reloadable — a new bean or template is a data change.
- **Prerequisites:** G1 (solver consumes it).
- **Feeds into:** M2G2 (the calibration tool edits it), M1G14 (search over it), M2G10 (ranking).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech: Web build:** Debug overlays + Tracy profiling to prove the WASM solver + chart update hold interactive latency under continuous slider input, Catch2 tests for the solver's numerical determinism and the vector-distance math, on a GitHub Actions matrix (native + Emscripten).
- **Prerequisites:** G1 (solver to instrument).
- **Feeds into:** M2G3 (CI hardening); the latency budget M2G1 validates on mobile web.

- **G5 2D Chart Rendering Core** — the extraction-yield curves + the sensory **taste radar** (custom-rendered, no chart libraries).

Tech & dependencies

- **Tech: Web build:** A custom chart renderer (axes, series, the multi-axis radar, batching) in OpenGL ES/WebGL via the WASM module, updated each frame from the solver output.
- **Prerequisites:** G1 (data to plot).
- **Feeds into:** M2G4 (advanced charting); the visualizer and the dial-in preview.

- **G9 Interaction Layer** — the live **sliders / brushing** that continuously re-inject parameters and recompute the radar.

Tech & dependencies

- **Tech: Web build:** Pan/zoom/brush/slider interaction that streams parameter changes into the solver each frame and updates the charts without frame drops; crosshair/linked-view hooks.
- **Prerequisites:** G1 (solver), G5 (charts).
- **Feeds into:** The real-time visualizer feel; the dial-in tool's live preview.

- **G14 Query & Filter Engine** — the multi-attribute catalog search + the **taste-vector ranking** foundation.

Tech & dependencies

- **Tech:** Incremental filtering over an indexed coffee databank + a SIMD vector-distance ranking of beans by similarity to the user's taste profile and past logs (hardware parallel vector instructions).
- **Prerequisites:** G2 (databank), G1 (taste coordinates).
- **Feeds into:** M2G10 (insight/ranking + the bounded neural re-rank); the relevancy search feature.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Profiling + SIMD/threading optimization of the WASM solver and the vector service, sanitizers, and validation on the mobile-web target (Pixel 9a browser); the Linux server (the C++ vector-search + neural re-rank service) set up (account + software + basic connectivity test) by M2.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).

- **Feeds into:** The web/mobile deliverable + a low-latency ranking service.
- **G2 ★ Content Editor — Core (Dial-In & Recipe Calibration Utility)** — author/tune bean dial-in templates + sensory envelopes; the closed-loop diagnostic previews the corrected radar before committing a versioned profile.
Tech & dependencies
 - **Tech:** A tool to author bean-specific templates (grind microns, water temp, ratio bounds, sensory target envelopes), ingest qualitative feedback (sour/astringent/hollow), run inverse heuristics to compute corrections, and preview the corrected radar; round-trip JSON with profile versioning to the databank.
 - **Libraries/Software:** the project's WASM solver + chart renderer, nlohmann/json, Firebase/DB client.
 - **Prerequisites:** G2 (template data), M1G1/M1G5 (solver + charts for preview).
 - **Feeds into:** Roaster/contributor content; the dial-in feature; the custom-tools showcase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
Tech & dependencies
 - **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, tests (solver determinism, vector math, data loading), and packaging of the Emscripten web build + the Linux vector service via GitHub Actions.
 - **Prerequisites:** G3 (CI & test foundation).
 - **Feeds into:** Release/showcase builds of every goal.
- **G4 Charting — Advanced** — multi-pane views, overlays and synced cursors (compare brews / beans side by side).
Tech & dependencies
 - **Tech: Web build:** Multi-pane charts, overlaying two brews' extraction curves / radars, synced cursors and comparison views built on M1G5.
 - **Prerequisites:** M1G5 (chart core), M2G1 (optimization).
 - **Feeds into:** The compare/diagnose experience at showcase.
- **G10 Insight Engine — Trends & Anomalies (Taste-Vector Ranking & Dial-In)** — the bean-ranking + the inverse-correction dial-in logic (no ML libraries; a bounded neural re-rank optional).

Tech & dependencies

- **Tech:** The taste-similarity ranking (from the SIMD vectors + user logs), the inverse-heuristic dial-in corrections (feedback → parameter deltas), and an optional bounded neural cross-encoder re-rank behind the C++ vector service; correctness verified against held-out logs.
- **Prerequisites:** M1G14 (vector search), M2G1 (the service).
- **Feeds into:** The recommendation + dial-in features at showcase.
- **G8 Firebase — Accounts, Sharing & Analytics** — accounts (B2C + B2B), the community databank/logs, and the shop/subscription store.

Tech & dependencies

- **Tech:** Firebase Auth (users + roasters), Firestore for community brew logs, shared profiles and shop/subscription/inventory data, and optional Analytics, via the API layer.
- **Prerequisites:** G2 (log/profile data model).
- **Feeds into:** The community + the (light) e-commerce; sharing versioned dial-in profiles.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — a data product lives on **UX and information design**: the explore/diagnose/recommend flows, the view/drill-down architecture, the metric/threshold (taste-model) design, the data storytelling and the shop/community UX are a large surface for two designers; the art surface is comparatively small (perceptual palette, chart styling, iconography, dense-view UI) and suits one artist (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the databank/UX outgrows two designers, that is the place to add, not art. *Scope note (advisory):* a WASM extraction solver + a SIMD vector service + custom charts + a dial-in tool + a community databank + a shop is a large surface — the solver + the taste radar + the catalog vector-search + the dial-in tool is the M1/M2 floor; the neural re-rank, the full community and the shop/subscription/B2B are the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (who uses BEANS — coffee newcomers vs enthusiasts vs roasters — the explore/estimate/recommend loops, trust & legibility of the taste estimates, feasibility)
- M1D2 Prototype — the core explore loop (input brew params → see estimated taste → get a recommendation/alternative)
- M1D3 User Research, Personas & Competitive Analysis (data-consumer personas + a teardown of coffee apps / brew calculators; findings into the FDD)
- M1D5 Interaction & Data-Density Spec (the slider/dial/brush conventions and legibility rules for the extraction curves + radar)
- M1D7 Metric & Threshold Design (which taste attributes matter, the extraction/strength defaults, the dial-in target envelopes designed with users)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D5 Playtesting & Iteration Program (test whether newcomers can dial in a better cup, triaged)

- M2D6 Vertical Slice — one question answered end to end (a bean + params → estimated taste → recommended correction → saved/shared profile)
- M2D8 Feedback Pass — clarity pass (density without chart junk; the dial-in correction moment unmistakable)
- **UI/UX M1** (graded by the UI/UX instructor):
 - M1U1 ★ UI Standards & Wireframe Baseline — the data-product screen-flow, control mapping and readability rules for dense chart views.
 - M1U2 UX Flows & Wireframes — catalog → bean/brew view → visualizer/dial-in → recommendations → shop, wireframed per persona.
 - M1U3 Accessibility & Readability Foundations — type sizes, contrast and a colorblind-safe perceptual palette (critical for the radar/curves).
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — catalog, visualizer, dial-in, recommendations, community and shop navigable unaided on the mobile-web target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished visualizer + dial-in UX tested with target users on the Pixel 9a browser.
 - M2U4 Onboarding & Tutorial Design — a guided first exploration that gets a newcomer to a first recommendation without jargon.
 - M2U5 Mobile UX Adaptation & Ergonomics — thumb-zone layout, safe areas and dense-view fit on the mobile-web target.
- **Art M1:** 1 artist (data-product visual language — palette, chart styling, iconography, dense-view UI)
 - M1A1 ★ Art Bible / Proof of Concept (the data-product visual language: perceptual palettes, chart styling, iconography; mockups = the visualizer view + a bean/catalog view)
 - M1A3 Environment, Props & VFX Concepts (chart/backdrop styling + highlight-VFX concepts — the dial-in correction highlight)
 - M1A4 UI Art Concepts & Typography (chart legends, control iconography, dense-view legibility)

- **Art M2:**

- M2A1 ★ Critical Asset Production & In-Engine Visual Quality (the palette system, chart styling and iconography completed in-engine)
- M2A4 UI Art — States & Feedback (filter/brush/dial-in/alert feedback, legend and control polish)
- M2A5 VFX — Critical Effects & Transitions (highlight effects — a dial-in correction landing, a notable taste match)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — light, functional audio present & working (subtle UI / confirmation / dial-in-success cues), suitable and correctly triggered (a lecturer waiver on BGM may apply for this data utility).
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the V&A floor adapted to a data utility (subtle interaction / recommendation / dial-in cues), correctly triggered and unobtrusive.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback functionality the build needs (multi-channel; a distinct dial-in-success cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the dial-in calibration utility), prototype demo, team post-mortem (wk14)